1. Values, Variables, Types, and Zero Values

01. Values, Variables, Types, and Zero Values

Phase: Phase 1 — CS fundamentals through Go
Estimated time: 1 day
Slug: `01-values-variables-types`

Learning objectives

- Declare variables with var and short declaration
- List basic types and their zero values
- Explain why every variable always has a defined value in Go
- Choose between typed and untyped constants

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Every Go value has a type and a zero value. That rule eliminates entire classes of undefined bugs and shapes how you design APIs and structs.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- `var x T` always initializes x to T's zero value — safe to read immediately.
- Short declaration `:=` infers types inside functions; `var` works at package scope too.
- Constants may be untyped and convert until forced into a concrete type.
- Zeros: numbers 0, bool false, string "", nil for pointers/slices/maps/chans/funcs/interfaces.
- Type is part of the value's identity: `int` and `int64` are different types.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-values, && cd /tmp/gw-values,
go mod init example.com/scratch
```

Then create `main.go`:

```go
package main

import "fmt"

func main() {
    var n int
    var s string
    var p *int
    ready := true
    const Max = 3
    fmt.Printf("n=%d s=%q p=%v ready=%v Max=%d\n", n, s, p, ready, Max)
}
```

Run it:

```
go run .
```

Expected output:

```
n=0 s="" p=<nil> ready=true Max=3
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Declare bare `var` for int, string, bool, *int; print them.
2. Use `:=` for a float64 and print its type with `%T`.
3. Assign untyped `const Max = 3` to both int and float64 variables.
4. Shadow a variable with `:=` in an inner block; print inner vs outer.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Values and types

Every value in Go has a **type**. Types answer: what operations are allowed? What size does it occupy? What is its zero value?

Common types: - Integers: `int`, `int64`, `uint`, `byte` (alias for `uint8`) - Floats: `float64` (prefer over `float32` unless you have a reason) - Boolean: `bool` (`true` / `false`) - String: `string` (immutable sequence of bytes, usually UTF-8)

```go
var age int = 30
var name string = "Ada"
ready := true          // short declaration; type inferred
const MaxRetries = 3   // untyped constant unless given a type
```

Zero values

If you declare a variable without assigning, Go sets the **zero value**: - numbers → `0` - bool → `false` - string → `""` - pointers/slices/maps/channels/functions/interfaces → `nil`

This removes a whole class of "undefined variable" bugs common in other languages.

```
var n int
fmt.Println(n) // 0, always safe to read
```

Memory intuition

A variable name is a label for a location in memory holding a value of a known type. `int` on a 64-bit machine is typically 8 bytes. You do not manage that manually — but knowing sizes helps later with slices, structs, and performance.

Failure modes to watch while coding

- Using `None`-like thinking: Go has `nil` only for reference types, not for ints/strings
- Shadowing variables with `:=` inside inner blocks accidentally
- Assuming uninitialized variables are dangerous — in Go they are zeroed

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Python uses None widely; Go ints/strings are never nil — only reference types use nil.
- `:=` can shadow outer variables in nested blocks — a frequent footgun.
- Uninitialized locals are safe in Go; Python raises NameError.
- No duck typing at assignment: types must match exactly (or convert explicitly).

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Zero values let you return empty structs without constructors for simple cases — but document meaning when zero is ambiguous (e.g. time.Time).

Practice checklist

Before the exercise, confirm you can:

1. Explain "Values, Variables, Types, and Zero Values" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Using `None`-like thinking: Go has `nil` only for reference types, not for ints/strings
- Shadowing variables with `:=` inside inner blocks accidentally
- Assuming uninitialized variables are dangerous — in Go they are zeroed

Exercises

Work in `exercises/01-values-variables-types/`. Read that folder's `README.md` before you code.

What you will build

Implement `ZeroInt()` by returning a bare `var` (no literal 0), and `Describe(n, label)` as `label=n`.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/01-values-variables-types/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What is the zero value of `string`? Of `*int`?
2. When must you use `var` instead of `:=`?
3. Why are zero values useful in APIs?

Further reading

- https://go.dev/ref/spec#The_zero_value
- https://go.dev/tour/basics